Tema:

API HTTP/REST y persistencia de datos

Docente:

Jenner Noel Che Méndez

Escuela:

Instituto Tecnológico de Chiná

Carrera:

Ingeniería en informática

Alumno:

Uriel Barrales Martínez

Semestre:

7 AI

Fecha:

2 de noviembre de 2026

Índice

2.1 Recursos, URI, métodos HTTP, encabezados, códigos de estado, JSON, idempotencia y restricciones REST....3

2.2. Diseño API-first con OpenAPI 3.1 o superior compatible....6

2.3. Validación de entrada, manejo uniforme de errores, serialización, paginación, filtrado, ordenamiento y búsqueda....8

2.4. Acceso a MySQL/MariaDB o PostgreSQL/Supabase....10

2.5. Separación por capas: rutas/controladores, casos de uso o servicios, dominio y acceso a datos....12

2.6 Consumo desde HTML/Bootstrap/JavaScript o Flutter/Dart....14

2.7 Pruebas de endpoints y colección reproducible de solicitudes....16

Referencias....18

2.1 Recursos, URI, métodos HTTP, encabezados, códigos de estado, JSON, idempotencia y restricciones REST

El diseño de interfaces de programación de aplicaciones basadas en el estilo arquitectónico de Transferencia de Estado Representacional, conocido como REST por sus siglas en inglés, constituye el estándar dominante para la construcción de sistemas distribuidos y servicios web interoperables. Formulada originalmente por Roy Fielding, esta arquitectura abstrae los elementos de información del dominio de negocio en el concepto fundamental de recurso.

Cada recurso debe ser direccionable de forma única mediante Identificadores Uniformes de Recursos, estructurados gramaticalmente a través de sustantivos en plural que reflejen entidades del mundo real, evitando de manera explícita la inclusión de verbos o acciones en las rutas de acceso, ya que la intención de manipulación se delega por completo a la semántica propia del protocolo de transporte HTTP.

Los métodos del protocolo HTTP proporcionan los verbos estandarizados que definen la naturaleza de las operaciones sobre los recursos. El método GET se destina a la recuperación segura de representaciones sin alterar el estado del servidor. El método POST se utiliza para la creación de subrecursos subordinados o el procesamiento de datos que generan un nuevo estado. El método PUT reemplaza de manera integral la representación existente de un recurso o lo crea en caso de que el identificador sea asignado directamente por el cliente.

El método PATCH realiza modificaciones parciales sobre campos específicos del recurso. El método DELETE remueve de forma lógica o física la entidad especificada. Adicionalmente, métodos como HEAD permiten verificar la presencia de recursos y obtener metadatos sin descargar el cuerpo de la respuesta, OPTIONS expone las capacidades de comunicación y políticas de origen cruzado admitidas, y la propuesta del método QUERY habilita la ejecución de consultas complejas mediante un cuerpo de solicitud sin perder la semántica de lectura.

Las operaciones dentro del protocolo HTTP se dividen formalmente según sus propiedades de seguridad e idempotencia. Un método se considera seguro cuando su ejecución se limita a la lectura de información sin producir efectos secundarios sobre el

estado del sistema. Por su parte, la idempotencia es una propiedad fundamental de los sistemas distribuidos que establece que la ejecución repetida de una misma petición debe producir exactamente el mismo efecto en el servidor que si se hubiera ejecutado una sola vez. Esta condición se expresa formalmente como: $f\big(f(x)\big) = f(x)$.

Bajo este principio, los métodos GET, HEAD, OPTIONS, PUT y DELETE son estructuralmente idempotentes, garantizando que reintentar una solicitud ante una falla de red no corromperá el estado ni creará registros duplicados. En contraste, el método POST no es idempotente, ya que cada llamada sucesiva creará una nueva entidad subordinada.

La transmisión de metadatos contextuales y la negociación del formato de intercambio se gestionan mediante los encabezados HTTP. El encabezado Content-Type informa el tipo de medio contenido en el cuerpo del mensaje, mientras que Accept permite al cliente especificar los formatos que está preparado para procesar, consolidando a JSON como el estándar de intercambio por su estructura liviana y su compatibilidad nativa con los lenguajes de programación modernos.

La autenticación y delegación de acceso se canalizan en el encabezado Authorization a través de esquemas como tokens de portador. Por otro lado, la concurrencia optimista y la validación de caché se coordinan mediante encabezados como ETag, If-Match y Cache-Control.

La comunicación del resultado de una petición se realiza mediante códigos de estado HTTP clasificados en categorías numéricas semánticas. La clase 2xx indica una resolución exitosa, destacando códigos como 200 OK para lecturas o modificaciones satisfactorias, 201 Created para la creación efectiva de un recurso, y 204 No Content cuando la operación culmina con éxito sin requerir la devolución de un cuerpo.

La clase 3xx gestiona la redirección de solicitudes. La clase 4xx identifica errores atribuibles al cliente, como 400 Bad Request por incorrecciones sintácticas, 401 Unauthorized por falta de autenticación, 403 Forbidden por insuficiencia de permisos, 404 Not Found cuando el recurso no existe, y 422 Unprocessable Entity cuando existen violaciones semánticas. Finalmente, la clase 5xx denota fallos internos en el servidor, como 500 Internal Server Error o 503 Service Unavailable.

Para que un sistema sea categorizado estrictamente como RESTful, debe cumplir de forma rigurosa las seis restricciones arquitectónicas enunciadas por Fielding.

La primera es la interfaz uniforme, que exige la identificación de recursos en la URI, la manipulación de recursos mediante representaciones, los mensajes auto-descriptivos y el principio HATEOAS, donde la respuesta del servidor incluye enlaces hipermedia que guían las acciones posteriores del cliente.

La segunda es la condición sin estado, la cual establece que el servidor no debe conservar el contexto de la sesión del cliente entre peticiones, obligando a que cada solicitud contenga la totalidad de la información requerida para ser procesada.

La tercera es la capacidad de almacenamiento en caché, que exige que las respuestas sean explícitamente etiquetadas como almacenables o no para optimizar el tráfico de red.

La cuarta es la estructura en sistema de capas, que permite intercalar intermediarios como balanceadores de carga, proxies o gateways de forma transparente.

La quinta es la separación estricta cliente-servidor, que aísla la interfaz de usuario de las preocupaciones de almacenamiento de datos.

La sexta es la restricción opcional de código bajo demanda, que permite al servidor extender temporalmente las funciones del cliente mediante la ejecución de scripts.

2.2. Diseño API-first con OpenAPI 3.1 o superior compatible

La metodología de desarrollo API-First sostiene que el diseño del contrato de la interfaz de programación de aplicaciones debe preceder a cualquier trabajo de implementación de código fuente en el backend o en el frontend. Este enfoque establece la especificación como la única fuente de verdad formal dentro del ciclo de vida del desarrollo de software.

Al consensuar los contratos de forma previa, las organizaciones consiguen desacoplar el trabajo de los diferentes equipos de ingeniería, permitiendo la generación automatizada de servidores de simulación, la construcción de kits de desarrollo de software cliente fuertemente tipados, el cumplimiento de pruebas de integración continuas y la publicación de documentación interactiva actualizada de forma permanente.

La especificación OpenAPI en sus versiones 3.1 y 3.2 representa la evolución más significativa del estándar para la descripción de APIs HTTP. El avance técnico más relevante de la versión 3.1 es su convergencia absoluta con el borrador 2020-12 de JSON Schema, superando la divergencia histórica en la que OpenAPI utilizaba un subconjunto modificado del esquema.

Esta alineación permite utilizar tipos de datos nativos para la representación de valores nulos mediante matrices de tipos, como el uso de cadenas y nulos combinados en lugar de la propiedad obsoleta de nulabilidad aislada. Asimismo, introduce soporte estricto para límites numéricos exclusivos, la validación de arreglos prefijados mediante tuplas y la incorporación de webhooks como objetos de primer nivel desacoplados de las rutas de consulta tradicionales.

En la estructuración de la especificación, los modelos de datos se definen mediante esquemas reutilizables que imponen restricciones de tipo, formato, patrones de expresiones regulares y valores requeridos. Los parámetros de la petición se clasifican según su ubicación en la ruta, la consulta, los encabezados o las galletas de sesión, detallando su obligatoriedad y reglas de conversión.

El mapeo de respuestas organiza cada código de estado HTTP con su correspondiente tipo de medio y esquema asociado, siendo indispensable incluir estructuras específicas para las respuestas de fallo. La declaración de muestras de datos mediante el objeto de

ejemplos permite proveer múltiples escenarios reales de uso con descripciones contextuales para enriquecer la documentación y alimentar las herramientas de prueba.

La evolución inevitable del dominio de negocio requiere la adopción de estrategias deliberadas de versionado para garantizar la estabilidad de los clientes existentes y evitar rupturas operativas. El versionado por ruta en la URI es la estrategia más extendida debido a su visibilidad explícita tanto en los registros de red como en las herramientas de desarrollo.

No obstante, existen alternativas como el versionado por parámetros de consulta, el uso de encabezados personalizados de versión, y la negociación de contenido avanzada a través de la especificación de tipos de medios propios dentro del encabezado Accept, la cual permite evolucionar la representación interna de los recursos sin alterar las URIs principales de la arquitectura.

2.3. Validación de entrada, manejo uniforme de errores, serialización, paginación, filtrado, ordenamiento y búsqueda

La gestión rigurosa del flujo de entrada y salida de datos es la base de la seguridad, la robustez y la predecibilidad en las aplicaciones web cliente-servidor. La validación de las peticiones debe estructurarse mediante una estrategia de defensa en profundidad organizada en dos etapas complementarias.

La primera etapa es la validación sintáctica o de forma, ejecutada en la capa adaptadora de entrada, cuyo propósito es verificar que la estructura JSON cumpla con los tipos de datos, los formatos estandarizados como identificadores únicos universales, direcciones de correo o fechas RFC 3339, y las restricciones de longitud requeridas.

La segunda etapa es la validación semántica o de negocio, desarrollada en los servicios de aplicación, cuya tarea es evaluar si los datos procesados son consistentes con el estado actual del almacén de datos y las reglas del dominio.

El tratamiento de las condiciones de fallo debe alejarse de respuestas arbitrarias e implementar estándares internacionales de comunicación de errores. La norma IETF RFC 9457 define una estructura JSON normalizada para la notificación de problemas en APIs HTTP servida bajo el tipo de medio específico para detalles de problemas.

Este estándar establece cinco miembros principales: una referencia URI que categoriza el tipo de problema, un título breve e inmutable que resume la clase de fallo, el código de estado HTTP idéntico al del encabezado, una explicación detallada e individualizada del motivo del error en la petición actual, y una instancia URI que actúa como identificador único de la ocurrencia para fines de trazabilidad y auditoría en los registros del sistema.

La norma promueve la extensibilidad mediante la adición de miembros secundarios para detallar violaciones específicas en campos individuales bajo el principio de procesamiento defensivo.

La serialización de salida y deserialización de entrada deben mantener reglas estrictas de consistencia en todo el ecosistema de aplicaciones. Las fechas y marcas temporales deben representarse inequívocamente bajo el estándar ISO-8601 expresado en tiempo universal coordinado. Los valores numéricos de alta precisión, como los montos financieros, deben

ser serializados como cadenas de texto cuando interactúen con entornos JavaScript para evitar la pérdida implícita de dígitos por el procesamiento de coma flotante de 64 bits. Asimismo, la convención de nomenclatura de las claves del cuerpo JSON debe fijarse de forma homogénea en toda la infraestructura de la API.

El acceso a colecciones masivas de información requiere la implementación de patrones de fragmentación de datos para preservar la memoria del servidor y el ancho de banda de la red. La paginación basada en desplazamiento y límite mediante sentencias relacionales simples es fácil de construir, pero presenta una degradación severa de rendimiento con una complejidad temporal expresada como: $O(N)$.

Además de mostrar inconsistencias cuando se producen inserciones o eliminaciones concurrentes durante la navegación. Por el contrario, la paginación basada en cursor o por clave utiliza un puntero inmutable que hace referencia al último elemento obtenido, ofreciendo un rendimiento constante de la forma: $O(1)$.

Garantizando la estabilidad del conjunto de resultados. El filtrado de datos debe canalizarse a través de parámetros de consulta estructurados, el ordenamiento mediante atributos explícitos con prefijos que indiquen la dirección descendente o ascendente, y la búsqueda de texto completo aislada en parámetros dedicados para aplicar índices especializados en la base de datos.

2.4. Acceso a MySQL/MariaDB o PostgreSQL/Supabase

La persistencia del estado de la aplicación recae en la integración con motores de bases de datos relacionales comprobados como MySQL, MariaDB, PostgreSQL o plataformas administradas como Supabase. El patrón Repositorio actúa como un componente mediador que abstrae los detalles de la infraestructura de almacenamiento, ofreciendo a las capas superiores una interfaz que simula una colección de objetos de dominio en memoria.

Esta abstracción aísla la lógica central del software de las sintaxis SQL específicas y de las librerías de mapeo objeto-relacional, permitiendo cambiar o evolucionar los mecanismos de persistencia sin impactar las reglas de negocio.

La preservación de la consistencia en entornos de acceso concurrente exige el uso de transacciones respaldadas por las propiedades de atomicidad, consistencia, aislamiento y durabilidad. La atomicidad garantiza que todas las sentencias dentro de un bloque transaccional se completen satisfactoriamente o se reviertan totalmente en caso de fallo.

La consistencia asegura que la base de datos transicione únicamente entre estados válidos que respeten las reglas de integridad. El aislamiento previene la interferencia entre transacciones simultáneas mediante la configuración de niveles como lectura comprometida, lectura repetible o serializable. La durabilidad asegura que las modificaciones confirmadas mediante un commit persistan de manera irreversible a pesar de caídas del suministro eléctrico o fallos del sistema operativo.

La evolución controlada de la estructura de la base de datos a lo largo del tiempo se realiza mediante herramientas de migración versionadas. Existen dos enfoques de diseño principales para la gestión de migraciones: el enfoque imperativo y el enfoque declarativo.

Las herramientas imperativas ejecutan una secuencia cronológica estricta de archivos SQL o cambios explícitos de versión para avanzar o retroceder la estructura. Las herramientas declarativas comparan un esquema de estado deseado expresado en archivos de configuración contra el estado físico actual del motor relacional, calculando y aplicando automáticamente el código diferencial necesario. El uso de estas herramientas

integrado en tuberías de despliegue continuo previene la deriva de esquema entre los entornos de desarrollo, pruebas y producción.

La base de datos relacional debe operar como la última línea de defensa en la integridad de los datos. Esto exige la definición explícita de restricciones de claves primarias, claves foráneas con reglas de acción referencial en borrado o actualización, restricciones de unicidad, comprobaciones de rango y la denegación de valores nulos.

En motores avanzados como PostgreSQL y plataformas como Supabase, la Seguridad a Nivel de Fila representa un mecanismo de control de acceso granular. A diferencia de las autorizaciones globales por tabla, las políticas de seguridad a nivel de fila evalúan de forma dinámica expresiones lógicas en cada fila individual para determinar si el usuario autenticado tiene permisos de lectura, inserción, actualización o eliminación.

Para mantener un rendimiento óptimo en la evaluación de estas políticas sobre tablas voluminosas, es una buena práctica envolver las funciones de contexto del sistema dentro de subconsultas simples, permitiendo que el optimizador de consultas ejecute la función una sola vez por petición en lugar de reevaluarla iterativamente por cada fila escaneada.

2.5. Separación por capas: rutas/controladores, casos de uso o servicios, dominio y acceso a datos

La organización de la arquitectura interna del software se fundamenta en la división de responsabilidades por capas delimitadas, siguiendo los principios de la Arquitectura Limpia y la Arquitectura Hexagonal.

Este diseño se rige por el Principio de Inversión de Dependencias, el cual dicta que los módulos de alto nivel que contienen las reglas esenciales del negocio no deben depender de los módulos de bajo nivel orientados a la infraestructura, la red o las bases de datos. En su lugar, todas las interacciones entre los componentes se realizan a través de abstracciones e interfaces, logrando un sistema altamente modular, mantenible y desacoplado de las tecnologías periféricas.

La capa de rutas y controladores se posiciona en el límite exterior de la aplicación como el adaptador de entrada de la interfaz HTTP. Su única responsabilidad consiste en escuchar las peticiones de red, extraer y deserializar los parámetros de la URI y los cuerpos JSON, ejecutar la validación sintáctica inicial de los datos recibidos y delegar el procesamiento a la capa interna.

Una vez recibida la respuesta o la excepción desde la lógica de la aplicación, el controlador se encarga de traducir dicho resultado al contrato HTTP correspondiente, empaquetando los fallos según las normas estandarizadas y retornando la respuesta al cliente.

La capa de casos de uso o servicios de aplicación constituye la capa intermedia encargada de orquestar los flujos de trabajo del sistema. Esta capa no contiene las reglas de negocio fundamentales ni conoce los detalles de la infraestructura web o del lenguaje de consulta a la base de datos.

Su propósito es coordinar la ejecución de las entidades del dominio, controlar los límites de las transacciones ACID, llamar a servicios complementarios de notificación o procesamiento, y administrar las interacciones con los puertos abstractos de persistencia.

El corazón irremplazable de la arquitectura reside en la capa de dominio. Esta capa concentra las entidades, los objetos de valor y los servicios de dominio que encapsulan la lógica y la semántica del negocio de la organización.

En la capa de dominio se definen las interfaces abstractas de los repositorios, estableciendo qué operaciones de almacenamiento requiere el negocio sin prescribir cómo se realizarán técnicamente. Al mantener esta capa pura y libre de dependencias de librerías externas, se garantiza que las reglas centrales puedan ser probadas en aislamiento total y se mantengan estables ante cambios tecnológicos.

La capa de infraestructura y acceso a datos se ubica en la periferia como el adaptador de salida. En esta capa se implementan de manera concreta las interfaces de repositorio declaradas por el dominio, utilizando tecnologías específicas como motores relacionales, clientes de bases de datos, sistemas de archivos o llamadas a servicios web de terceros.

Esta inversión de dependencias logra que los detalles técnicos dependan de las abstracciones del dominio y no a la inversa, permitiendo sustituir o actualizar componentes de infraestructura sin modificar la lógica interna de la aplicación.

2.6 Consumo desde HTML/Bootstrap/JavaScript o Flutter/Dart

El consumo de interfaces de programación RESTful desde clientes web construidos con HTML, Bootstrap y JavaScript, o desde aplicaciones móviles desarrolladas con el lenguaje Dart sobre el marco Flutter, requiere un diseño defensivo para la gestión de operaciones asíncronas.

Debido a la latencia inherente a las comunicaciones de red y a la posibilidad de fallos parciales, la interfaz de usuario debe actuar como una máquina de estados finitos que represente de forma explícita e inmediata la situación real de la transferencia de información, ofreciendo retroalimentación visual clara que prevenga la incertidumbre del usuario.

Toda interfaz orientada a la interacción con APIs HTTP debe estructurar la presentación visual alrededor de cuatro estados fundamentales del ciclo de vida asíncrono. El estado de carga se activa de forma instantánea al iniciar la petición a la red, procediendo a bloquear las acciones repetidas en la pantalla y mostrando componentes de progreso o maquetas de contenido conocidas como pantallas esqueleto.

El estado de error se desencadena al capturar excepciones de conexión o respuestas HTTP con códigos de la serie 4xx o 5xx, procesando la carga útil del fallo para desplegar mensajes informativos comprensibles e interactivos con opciones de reintento. El estado vacío se manifiesta cuando la API responde con un código de éxito 200 OK pero la colección retornada carece de elementos, requiriendo una vista ilustrativa que oriente al usuario a realizar su primera creación de datos.

Finalmente, el estado de éxito se alcanza al recibir y deserializar correctamente la información esperada, procediendo a renderizar los componentes definitivos de la interfaz gráfica.

En el desarrollo web basado en JavaScript nativo y hojas de estilo de Bootstrap, la invocación de servicios se realiza mediante la interfaz Fetch y la sintaxis de funciones asíncronas. La gestión de los estados visuales se efectúa conmutando clases de utilidad para ocultar o desplegar contenedores del documento HTML, manipulando elementos de alerta o barras de progreso en función de la resolución o rechazo de las promesas de red.

En el entorno de desarrollo de Flutter con el lenguaje Dart, la gestión del flujo asíncrono se implementa de manera declarativa mediante patrones de arquitectura de estado como Riverpod o BLoC. En el marco de Riverpod, la clase sellada de valores asíncronos proporciona una representación fuertemente tipada de las distintas fases de la petición a través de variaciones de datos, carga y error.

La integración de estos estados dentro del árbol de componentes visuales se realiza mediante coincidencia de patrones con el método when, el cual impone en tiempo de compilación el manejo obligatorio de todas las alternativas de la interfaz:

```
return asyncAccounts.when(
  data: (accounts) => accounts.isEmpty
      ? const EmptyView()
      : AccountList(accounts: accounts),
  loading: () => const ProgressIndicator(),
  error: (err, stack) => ErrorTile(error: err),
);
```

La separación entre la interfaz de usuario y la lógica de infraestructura se conserva delegando las peticiones a controladores de estado dedicados, los cuales capturan las excepciones de la API HTTP y las empaquetan en instancias de error para su renderizado defensivo.

2.7 Pruebas de endpoints y colección reproducible de solicitudes

La garantía de calidad de una interfaz de programación de aplicaciones RESTful requiere la implementación de un esquema de validación automatizado que opere en diferentes niveles de aislamiento y cobertura. La estrategia de pruebas para servicios web se organiza en una estructura piramidal que comprende tres niveles principales. Las pruebas unitarias evalúan el comportamiento aislado de las entidades de dominio y los servicios de aplicación utilizando dobles de prueba para sustituir la persistencia.

Las pruebas de integración de endpoints verifican el funcionamiento conjunto del canal HTTP completo, evaluando la resolución de rutas, el procesamiento de middlewares, las reglas de validación, la serialización de datos y el acceso efectivo a instancias de bases de datos relacionales en entornos controlados. Por último, las pruebas de contrato garantizan de forma estricta que las respuestas reales emitidas por la implementación del software se mantengan en total cumplimiento con la especificación OpenAPI publicada.

Para asegurar la reproducibilidad de las pruebas de integración entre los miembros del equipo y dentro de las tuberías de integración continua, se utilizan estándares para la definición de colecciones de solicitudes HTTP.

Las herramientas como Postman e Insomnia permiten estructurar colecciones de peticiones en formatos como JSON v2.1, organizando llamadas parametrizadas mediante variables de entorno, scripts de pre-solicitud para la obtención automatizada de tokens de seguridad, y funciones de validación escritas en código ejecutable que verifican códigos de estado, tiempos de respuesta y esquemas de carga útil.

Una alternativa moderna al almacenamiento monolítico en formato JSON la ofrecen herramientas como Bruno y los archivos de extensión HTTP, los cuales almacenan cada petición en archivos de texto plano estructurados dentro del propio repositorio de código fuente de la aplicación.

Este enfoque orientado a texto plano se integra de forma transparente con los sistemas de control de versiones distribuidos como Git, eliminando los conflictos complejos de fusión de cambios que ocurren con frecuencia al colaborar en archivos JSON de gran tamaño.

La ejecución automatizada de estas colecciones dentro del ciclo de integración y despliegue continuo se efectúa mediante ejecutores de interfaz de línea de comandos como Newman o Bruno CLI. Integrados dentro de los flujos del servidor de construcción, estos ejecutores lanzan las colecciones de prueba contra instancias efímeras de la aplicación inicializadas mediante contenedores de software.

Cualquier discrepancia en las respuestas, fallo en los códigos de estado o violación del contrato de la API provoca de inmediato el rechazo del cambio y bloquea la promoción del código a los entornos de producción.

Referencias

Celis, J. A., & Gómez, L. M. (2021). Diseño e implementación de APIs RESTful con Node.js y bases de datos relacionales. Revista de Ingeniería y Sistemas, 15(2), 45-62.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Tesis doctoral, University of California, Irvine).

Fowler, M. (2018). Patrones de arquitectura de aplicaciones empresariales. Editorial Anaya Multimedia.

Hernández, V. R., & Martínez, P. E. (2023). Desarrollo de aplicaciones móviles y web: Integración de servicios REST con Flutter y JavaScript. Editorial Marcombo.

Richardson, L., & Ruby, S. (2012). RESTful Web Services: Servicios web para el mundo real (Trad. J. M. García). O'Reilly Media / Anaya Multimedia.